SOFTWARE ENGINEERING / CASE STUDY

NetSecurity Toolkit AI-Assisted Development Case Study

How I directed, tested, corrected, and documented an iterative Python security-toolkit project with ChatGPT as a development assistant.

Dan LeBlond

Executive summary

I used ChatGPT to help build **NetSecure Toolkit**, my Python networking and security project. It includes an HTTP tool, a port scanner, FTP and SSH connections, and examples of SQL injection. ChatGPT was a coding assistant I could ask questions and work through problems with.

What I did: I decided how the program should work, shared code and error messages, tested changes on my computer, and told ChatGPT when something still needed fixing. **What AI did:** It helped me understand technical topics, update Python code, find causes of errors, and write project documentation.

How we worked: I asked for a change, tried it, shared the result, and refined the solution. For example, I chose to keep `netscan.py` as the existing controller. ChatGPT helped make a simple menu that calls it, and we later worked through a file-path error that stopped the SQL demos from opening.

What the collaboration looked like

WHAT I DID	HOW CHATGPT HELPED	EXAMPLE
Chose to reuse my existing project code	Helped build a new menu that calls `netscan.py`	Menu choice 5 opens the SQL demos
Ran the program and shared the errors I saw	Explained error messages and suggested changes	Corrected a file-path problem when launching SQL demos
Asked what security fixes really changed	Explained risks and helped write clear notes	Documented why parameterized SQL queries are safer

01 / CONTEXT & DIVISION OF WORK

The project and the human–AI workflow

My assignment was to build a set of Python tools for networking and security practice. The program can make basic web requests, scan ports, connect to FTP and SSH servers, and show examples of unsafe and safer code. I kept these tools in separate files and added a numbered menu so a user can choose what to run. The code calls the project **NetSecure Toolkit**.

Architecture selected during the conversation

```
main.py       -> shows the numbered menu
netscan.py    -> starts the selected tool
modules/      -> contains the tools themselves
docs/Phase 4/ -> security review and examples
```

I wanted a simple **"controller controlling another controller."** At first, ChatGPT suggested calling each tool directly. I explained that `netscan.py` already knew how to run the tools, so the new menu should call that existing file instead. ChatGPT adjusted the design to follow my decision.

How I used ChatGPT across the six phases

PHASE	PROJECT WORK AND AI HELP
1 - HTTP	Built a basic web-request tool with Python sockets, plus a client/server exercise. ChatGPT helped explain how to run it and add it to the final menu.
2 - Scanner	Built a tool that checks a range of ports. When a scan looked stuck, I asked ChatGPT why and checked the final result.
3 - FTP / SSH	Built features to list/download FTP files and connect to SSH servers. ChatGPT helped design demo options and custom input in the menu.
4 - Security review	Reviewed unsafe Python examples and documented changes. I challenged ChatGPT when it described an incomplete security fix too confidently.
5 - SQL examples	Built an intentionally vulnerable login and a version using safer SQL queries. ChatGPT helped explain the difference and write documentation.
6 - Main menu	I chose to reuse `netscan.py` instead of rewriting the tools. ChatGPT helped build the menu and troubleshoot problems launching the demos.

Some early parts of the chat were not fully available, so I cannot assign every earlier line of code to a specific AI exchange.

Prompting as a practical engineering skill

I gave ChatGPT instructions about what to change, what to leave alone, and what I wanted explained. Below are real examples from my prompts, lightly edited for spelling and readability without changing their meaning.

1 Control the sequence of work

"Don't change anything in the code yet. Talk to me here in chat first."

I asked for discussion before editing the command-injection feature. **Skill:** stage a technical task into understanding, design, then implementation.

2 Protect completed functionality

"I think the login part is done, so don't touch that anymore."

I marked a completed area out of scope. **Skill:** give an AI coding assistant precise change boundaries to reduce regressions.

3 Challenge incomplete security claims

"Can you point out that this isn't a complete fix?"

I pushed for an honest limitation statement about `pickle.loads()`. **Skill:** assess whether a mitigation really removes the underlying risk.

4 Specify a preferred architecture

"I'm making a menu controller that calls the things I've already built in netscan.py."

I redirected the proposed implementation toward reuse of existing code. **Skill:** supply architecture, scope, and simplicity constraints rather than accepting the first solution.

5 Turn feedback into an acceptance test

"Use the examples I already have, and add an option for users to enter their own information with input checks."

I described a repeatable user experience: presets plus custom input and checks. **Skill:** express testable behavior in plain language.

Real troubleshooting episodes

Case A A dependency was installed, but the program still crashed

Observed: `ModuleNotFoundError: No module named paramiko` occurred when the menu launched `netscan.py`. The terminal showed `paramiko` installed for Python 3.12, while VS Code was using Python 3.13.

Human input: I pasted the actual installation output and debugger command, not just "it does not work." **AI assistance:** ChatGPT identified the interpreter mismatch and supplied a Python-3.13-specific installation command. **Observed result:** the next pasted log showed successful installation, followed by a working HTTP request through the menu.

> **Why it matters**
> This is evidence of diagnosing an environment problem from a traceback and package path—not changing unrelated application code until the actual cause was understood.

Case B Fixing one file path exposed another path dependency

Observed: Starting `main.py` from the user's home directory caused the child process to look for `netscan.py` in the wrong folder. I reported the exact command and error. ChatGPT updated the launcher to build an absolute path from `__file__`.

Regression: SQL menu option 5 still failed because `netscan.py` internally launched `modules/sql_demo.py` using a relative path. I pasted the new failure ("5 doesnt work anymore"), and the fix was revised to pass the project folder as the subprocess working directory.

```
project_folder = os.path.dirname(__file__)
netscan_path = os.path.join(project_folder, "netscan.py")
subprocess.run([...], cwd=project_folder)
```

Outcome supported: The user later reported that everything worked. This shows a full debugging loop and a correction to an earlier AI-generated fix; it is not independent verification of every feature.

Using AI to understand—not just copy—security changes

Phase 4: compare safer behavior with the underlying risk

For the instructor-provided vulnerable program, I asked questions about what each feature should be allowed to do before accepting a change. The discussion covered command allowlists, limited login/command attempts, handling bad input, file-writing restrictions, and binding a demonstration listener to localhost instead of a network-facing address. I also asked for comments explaining *why* changes reduced risk.

A particularly important exchange concerned insecure deserialization. A draft included blank-input checks, length limits, exception handling, and post-load type checks. I pushed for the README to say clearly that this was **not a complete fix**: untrusted `pickle.loads()` can execute code *before* a type check. The documented preferred real-world approach is not to deserialize untrusted pickle input.

Phase 5: contrast a vulnerable query with a parameterized one

VERSION	CODE EXAMPLE	WHAT IT SHOWS
Vulnerable login	SQL query built by inserting form text into a formatted string	Input can alter the SQL expression
Mitigated login	`... WHERE username=? AND password=?` `execute(query, (username, password))`	Values are supplied as data, rather than SQL syntax

I requested clearer, easier-to-understand explanations and later asked that the two changed lines be emphasized in the secure version. That documentation work helped explain the actual security change, rather than just calling one file “secure.”

What was actually observed in the pasted output

EVIDENCE	SUPPORTED CONCLUSION	BOUNDARY
HTTP response with “200 OK”	Raw-socket HTTP menu route connected and parsed a real response	Not a test of TLS or HTTPS
Scan finished: 0 open ports	The loop completed and reported a result	Does not imply a vulnerability was found
Flask launch, SQL query printed, POST /login	SQL demo ran and received a request with an injection-style input	The excerpt does not show the returned login result
User confirmation after path revisions	Developer reported features working	No automated or independent end-to-end test report provided

What this experience demonstrates

TRANSFERABLE SKILL	CONCRETE BEHAVIOR FROM THE RECORD
AI-assisted coding	Used ChatGPT to improve code and add a menu without rewriting existing tools
Prompt refinement	Gave clear instructions about what to change and what to leave alone
Debugging with tracebacks	Shared actual errors and terminal results to find the cause of problems
Technical decision-making	Decided how the files, menus, and project phases should work
Critical evaluation	Questioned unsafe suggestions and tested fixes that initially failed
Communication	Wrote simple instructions explaining how to use the program